АВТОРИЗАЦІЯ І УПРАВЛІННЯ ДОСТУПОМ

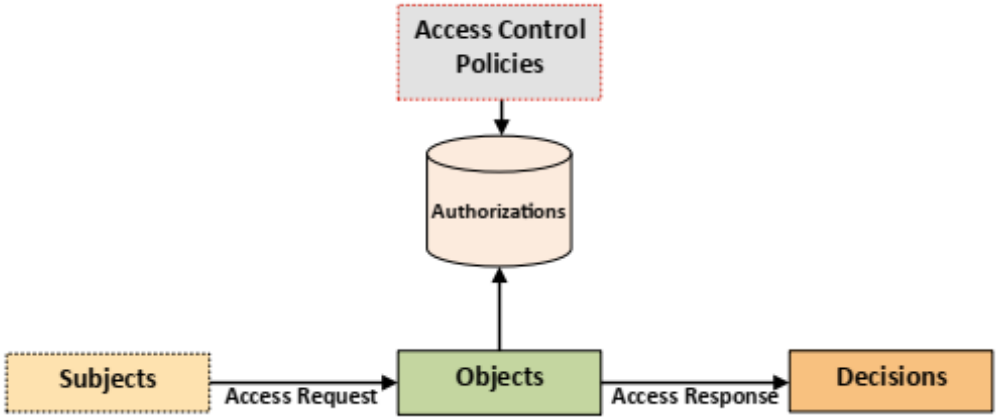
AAA

управління доступом

► **Управління доступом** - це механізм безпеки, який керує процесом взаємодії користувачів з системами і ресурсами, а також систем між собою.

► **Доступ** - це потік інформації між суб'єктом і об'єктом. **Суб'єкт** - активна сутність, яка запитує доступ до об'єкта або даними всередині об'єкта. Суб'єктом може бути користувач, програма або процес, який використовує доступ до об'єкту для виконання свого завдання. **Об'єкт** - пасивна сутність, яка містить інформацію.

► **Авторизація** - це процедура, в якій автентифікованому користувачеві присвоюється набір повноважень, який визначає його можливість отримати доступ до даних використовуючи додаток, а також можливість виконання окремих функцій програми.



управління доступом

Що таке контроль доступу?

- **Авторизація** - це процес, за допомогою якого система визначає, чи має конкретний користувач доступ до ресурсу
- **Дозвіл** : Відображає тільки поведінку програми
- **Права**: Що користувачеві насправді дозволено робити
- **Суб’єкт/Користува**ч: Кому/що ви надаєте право
- **Неявна роль**: Іменований дозвіл, пов'язаний з користувачем

```
if (user.isRole("Менеджер"));
```

- **Явна роль**: Іменований дозвіл, пов'язаний з ресурсом

```
if (user.isAuthorized("report:view:3324");
```

Порівняння дозволів, прав і привілеїв (Permissions, Rights, Privileges)

► **Дозволи** (Permissions) Загалом, дозволи стосуються доступу, наданого **об'єкту**, і визначають, що з ним можна робити. Якщо у вас є дозвіл на читання файлу, ви зможете відкрити його та прочитати. Ви можете надати користувачеві дозволи на створення, читання, редагування або видалення файлу на файловому сервері. Аналогічно, ви можете надати користувачеві права доступу до файлу, тому в цьому контексті **права доступу та дозволи є синонімами**. Наприклад, вам можуть бути надані дозволи на читання та виконання файлу програми, що дає вам право запускати програму. Крім того, вам можуть бути надані права на дані в базі даних, що дозволить отримувати або оновлювати інформацію в базі даних.

► **Право** (Rights) Право в першу чергу відноситься до можливості здійснювати дію щодо об'єкта з боку **користувача**. Наприклад, користувач може мати право змінювати системний час на комп'ютері або право відновлювати дані з резервних копій. Це тонка відмінність і не завжди підкреслена. Однак ви рідко побачите права вжити заходів щодо системи, які називаються дозволом.

► **Привілеї** (Privileges) Привілеї – це **поєднання прав і дозволів**. Наприклад, адміністратор комп'ютера матиме повні привілеї, якщо надали адміністратору повні права та дозволи на комп'ютер. Адміністратор зможе виконувати будь-які дії та отримати доступ до будь-яких даних на комп'ютері

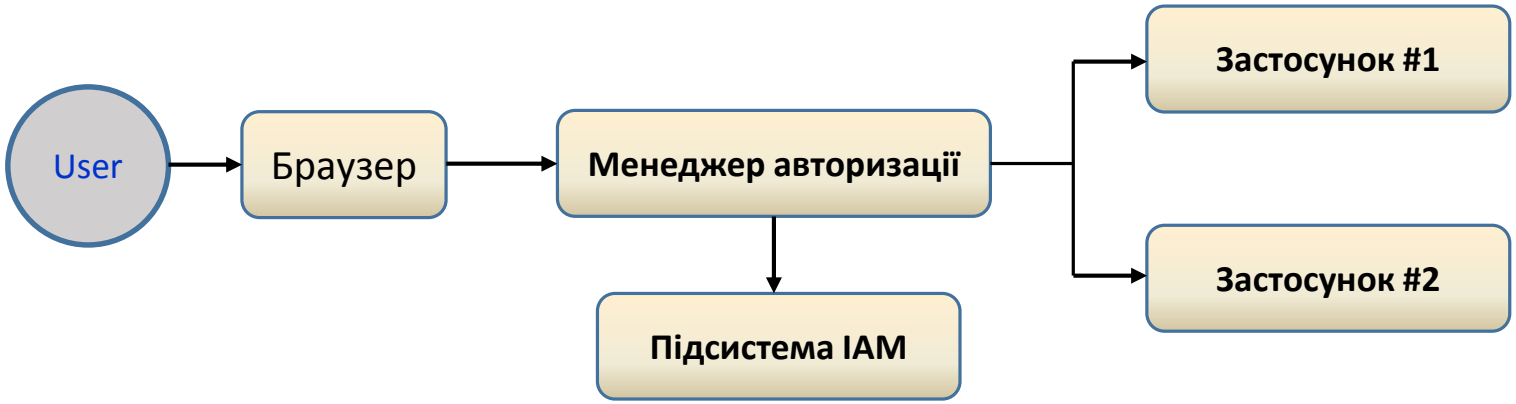
Авторизація

- Пов'язування прав доступу з однозначно визначеними особами за допомогою унікальних ідентифікаторів.
- Створення запису прав доступу, наданих ідентифікатору користувача для доступу ресурсів
- Застосування принципу найменших привілеїв
- Призначення індивідуальних привілеїв доступу до ресурсів на основі рівнів інформаційної безпеки та класифікації інформації.
- Визначення вимог щодо закінчення терміну дії прав привілейованого доступу.
- Заборона повторного використання ідентифікаторів. Це означає видалення записів авторизації, пов’язаних з ідентифікатором користувача, коли особа, якій призначено цей ідентифікатор користувача, змінює ролі або залишає організацію.

Рівні застосування політики авторизації

Рівні застосування політики авторизації

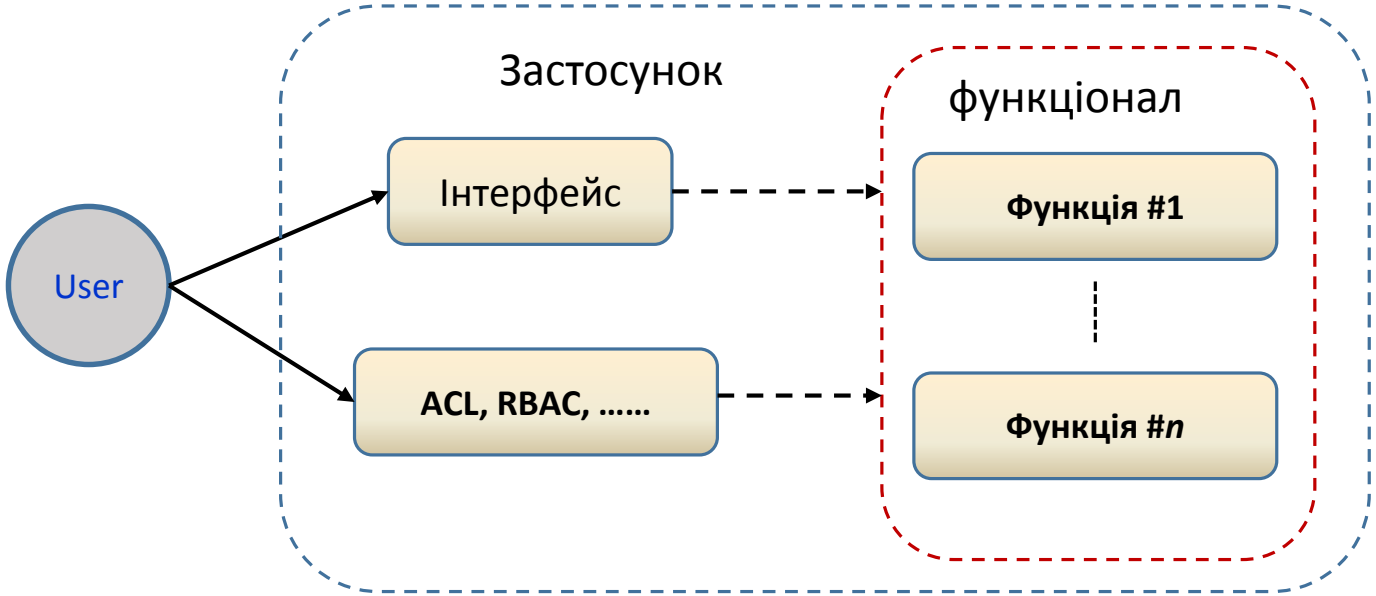
1. Доступ користувача до програми або API



На найвищому рівні авторизація контролює, чи має суб'єкт дозвіл на доступ до програми або API взагалі. Цей варіант використання часто зустрічається у корпоративному середовищі. Наприклад, співробітник відділу маркетингу немає права доступу до корпоративної бухгалтерської системі.
Цей рівень застосування політики авторизації може бути реалізований усередині застосунка або зовнішніми по відношенню до застосунку компонентами. Такі системи можуть виступати як високорівнева точка відхилення користувачів, які не авторизовані для доступу до програми взагалі.

Рівні застосування політики авторизації

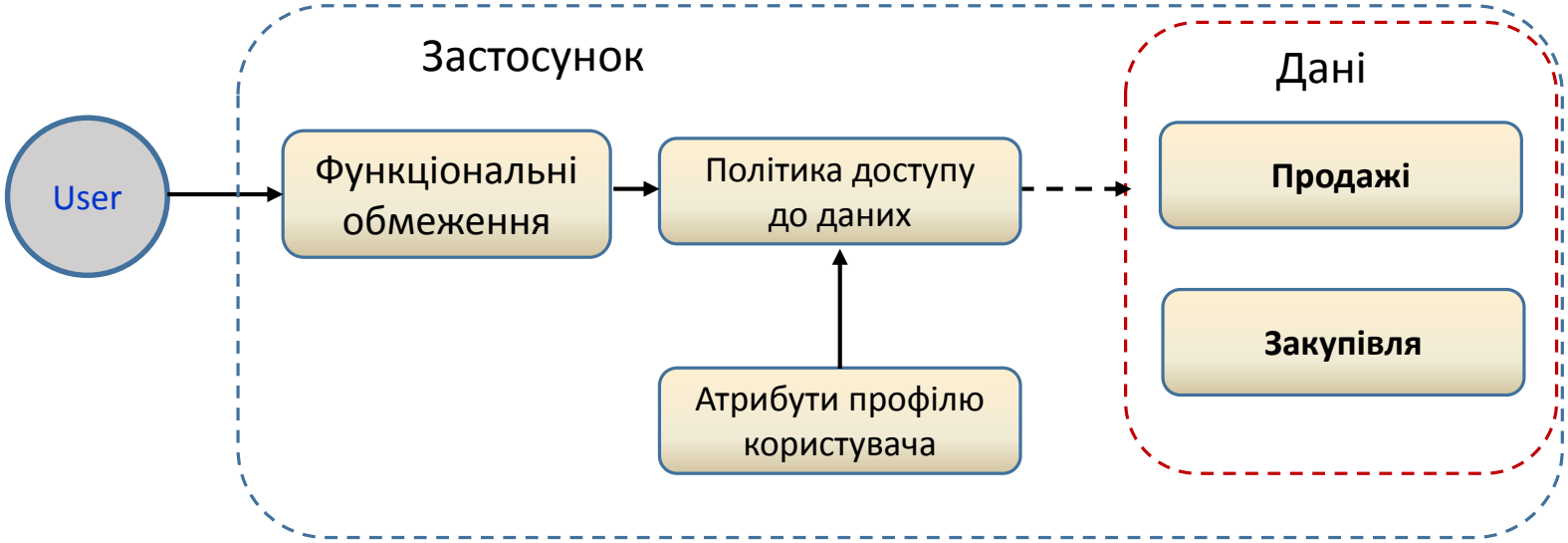
2. Функціональний доступ



Авторизація на функціональному рівні визначає, що суб'єкт може робити у додатку або API. Наприклад, молодший співробітник бухгалтерії у фінансовому відділі може мати доступ до корпоративної системи бухгалтерського обліку та вводити окремі журнальні записи, але не виконувати закриття місяця. Цей рівень авторизації та застосування політики доступу, як правило, залежить від конкретної програми. Він може використовувати інформацію про користувача, що зберігається в інших місцях, наприклад, ролі або групи в службі каталогів, але найчастіше він реалізується всередині програми або API.

Рівні застосування політики авторизації

3. Доступ до даних

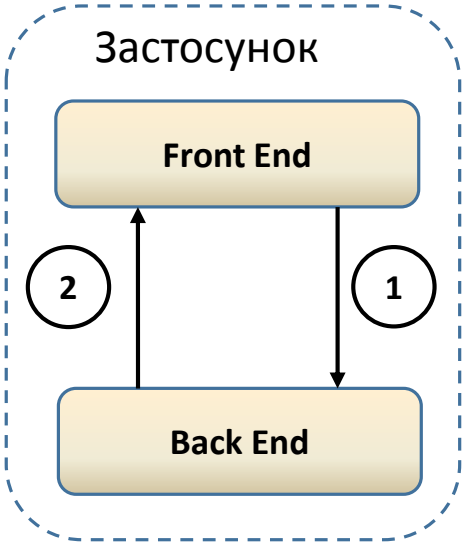


Третій рівень застосування політики авторизації та доступу регулює доступ до певних підмножин даних. Якщо політика доступу на функціональному рівні визначає функції, які може виконувати об'єкт, застосування політики доступу на рівні даних ще більше обмежує доступ до певних даних.

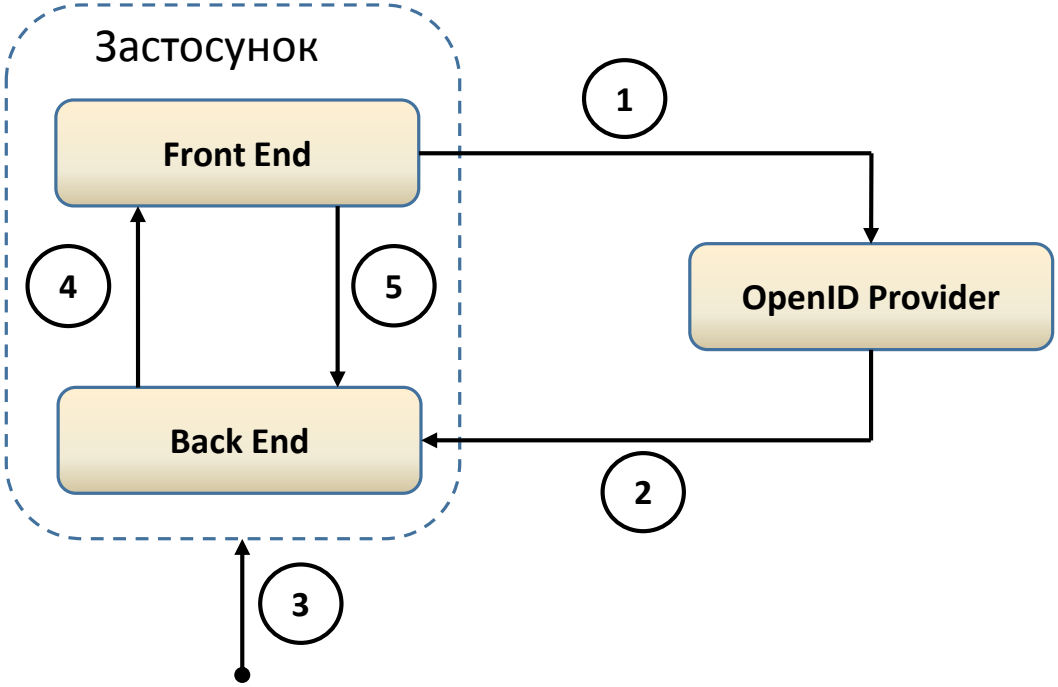
Наприклад, у додатку для введення замовлень на продаж користувач з роллю "регіональний менеджер з продажу" може мати право на функціональному рівні переглядати замовлення на продаж, але політика доступу на рівні даних обмежує їх до певного регіону, вказаного в атрибуті "Регіон продажів" профілю користувача.

На рівні даних доступ може бути забезпечений у додатку або API або **на базовому рівні зберігання даних (СУБД)**

Зовнішнє визначення автентифікаційних атрибутів користувача (OIDC)

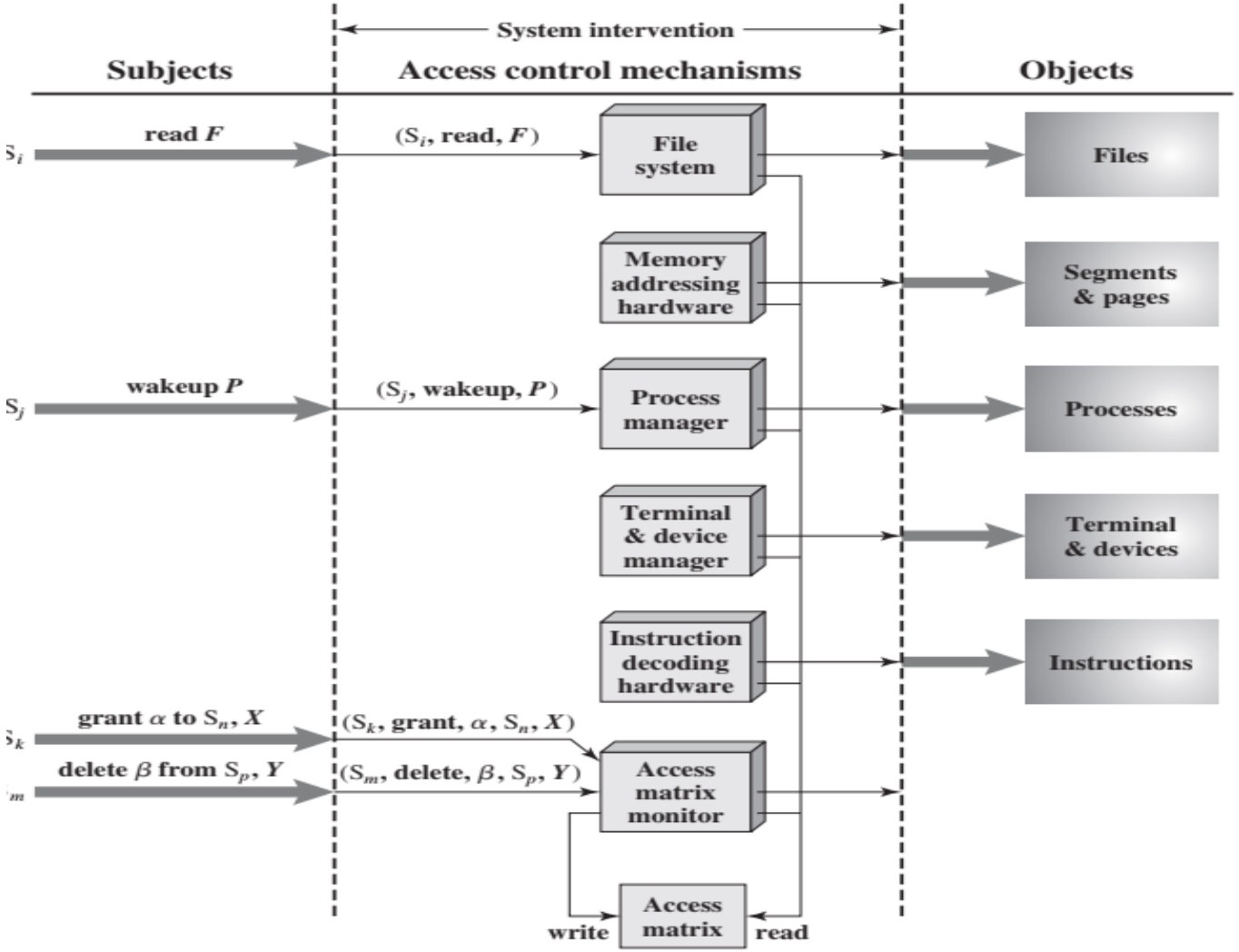


1.Сервер виконує авторизацію користувача.
2.UI відповідно до атрибутів авторизації або реалізація функціоналу застосунку



1. Перенаправлення до OpenID provider
2. ID Token суб'єкта (+атрибути авторизації)
3. Сервер виконує перевірку контролю доступу. Суб'єкт має доступ до функціоналу
4. UI відповідно до атрибутів авторизації
5. Реалізація функціоналу застосунку

Складність реалізації управління доступом



Моделі управління доступом

- **Модель управління доступом** - це структура, яка визначає порядок доступу суб'єктів до об'єктів. Для реалізації правил і цілей цієї моделі використовуються технології управління доступом і механізми безпеки.

Існує три основних моделі управління доступом:

- **дискреційна -** Discretionary Access Control (**DAC**)
- **мандатна** - Mandatory Access Control (**MAC**)
- **ролева -** Role-Based Access Control (**RBAC**)

Додаткові моделі управління доступом:

- **атрибутна** - Attribute-Based Access Control (**ABAC**)
- Capability-Based Access Control (**CapBAC**)
- Protocol-Based Access Control (**ProBAC**)
- Hybrid Access Control (**HyBAC**)

Моделі управління доступом

Дискреційне управління доступом

► Контроль доступ на основі принципу **володіння об’єктом**

► Система, яка використовує **діскреціонное (виборче) управління доступом** (DAC - Discretionary Access Control) дозволяє **власнику** ресурсу визначати, які **суб'єкти** можуть використовувати цей ресурс. Ця модель називається дискреційною (виборчої), тому що управління доступом засновано на рішеннях власника.

► Власники даних можуть передавати **право власності** на інформацію іншим користувачам.
► Власники даних можуть визначати **тип доступу**, що надається іншим користувачам (читання, запис, копія та т. Д.)
► Суб'єкт, якому дозволено дискреційний доступ до ресурсу, може **передати свої права доступу** іншому суб'єкту
► DAC має **недолік** - адміністратори не можуть централізовано керувати цими дозволами на файли / інформацію, що зберігаються на сервері

Дискреційне управління доступом

► Найчастіше модель DAC реалізується за допомогою списків контролю доступу (**ACL**), вміст яких визначено власниками. Робота ACL реалізуються **засобами операційної системи**.

► ACL - це табличне представлення матриці контролю доступу в який суб'єкті зіставляються з конкретними ресурсами, що належать до конкретних об'єктів.

► **Матриця доступу**

$$\{S, O, AR\}$$

де
S - набір суб'єктів
O - об'єкти
AR - набір прав доступу {*read*, *write*, *execute*}.

Список доступу, профіль користувача

Об'єкт / Суб'єкт	O_1	O_2	O_3	O_4 (Принтер)
S_1	read			
S_2				Print
S_3		read	Execute	
S_4	Read, write		Read, write	

можна забезпечити детальний контроль доступу, але це важко із **збільшенням масштабу системи** (тобто, де суб'єкти та їх права доступу до конкретних об'єктів збільшуються)

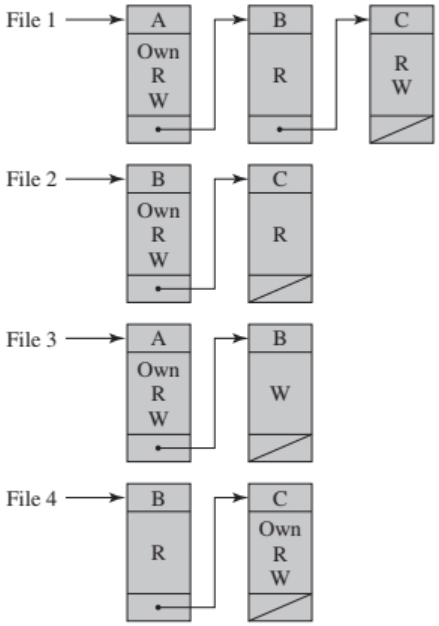
Дискреційне управління доступом

На практиці матриця доступу зазвичай є розрідженою і реалізується шляхом розкладання одним із двох способів.

► Матриця може бути **розкладена за стовпцями**, отримуючи списки контролю доступу (ACL); Для кожного об’єкта ACL перелічує користувачів та їх дозволені права доступу. ACL може містити запис за замовчуванням. Це дозволяє користувачам, які явно не вказані як такі, що мають спеціальні права, мати набір прав за замовчуванням. Набір прав за замовчуванням завжди повинен відповідати правилу найменших привілеїв. Елементи списку можуть включати як окремих користувачів, так і групи користувачів

		OBJECTS			
		File 1	File 2	File 3	File 4
	User A	Own Read Write		Own Read Write	
SUBJECTS	User B	Read	Own Read Write	Write	Read
	User C	Read Write	Read		Own Read Write

(a) Access matrix



(b) Access control lists for files of part (a)

Однак така структура даних не зручна для визначення прав доступу, доступних конкретному користувачеві.

Дискреційне управління доступом

► **Приклад для Windows**

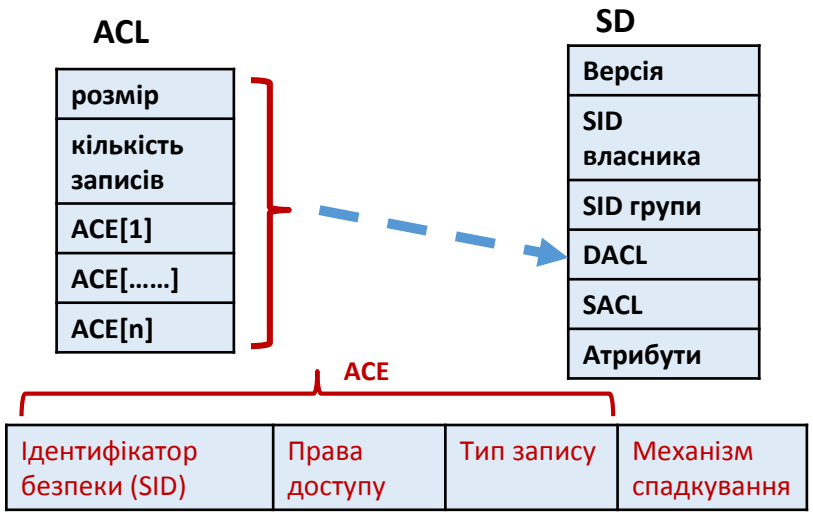
Після автентифікації користувача процесом Winlogon , всі процеси, запущені від імені цього користувача будуть ідентифікуватися спеціальним об'єктом, званим **маркером доступу** **(access token)** . Якщо процес користувача запускає дочірній процес, то його маркер успадковуються, тому маркер доступу уособлює користувача для системи в кожному запущеному від його імені процесі.

► **Узагальнена структура маркера доступу**.

SID користувача | **SID1 ...SIDn (***Ідентифікатори груп користувача***) | DACL** *за Замовчуванням* **| привілеї |Інші параметри**

► **Створення маркеру доступу** API-функції **CreateToken, CreateRestrictedToken**

Моделі управління доступом



Авторизація Windows заснована на зіставленні **маркера доступу суб'єкта** з **дескриптором безпеки об'єкта**.

► Якщо список ACE в DACL об'єкту порожній, до нього немає доступу ні у одного користувача (тільки у власника на зміну DACL).

► Якщо відсутній сам DACL в SD об'єкта - повний доступ до нього мають всі користувачі.

► Якщо який-небудь потік запросив доступ до об'єкта, підсистема SRM здійснює перевірку прав користувача, що запустив потік, на даний об'єкт, **переглядаючи його список DACL**. Перевірка здійснюється до появи дозволяють прав на **всі запитані операції**.

► Якщо зустрінеться забороняюче правило хоча б на одну запитану операцію, доступ не буде надано.

► Необхідно відзначити, що забороняюче правило поміщено в списку DACL завжди розміщуються перед вирішальними, тобто є **домінуючими** при перевірці прав доступу.

Моделі управління доступом

► Модель **мандатного управління доступом** (MAC - Mandatory Access Control) зазвичай підходить для надзвичайно захищених систем, включаючи захищені військові додатки або додатки для критично важливих даних.

► Тільки адміністратори, а не владники даних, вносять зміни до мережі безпеки.

► Всім даним визначається рівень безпеки, який відображає їх відносну чутливість, конфіденційність та значення захисту.

► У MAC користувачі і власники даних не можуть самостійно визначати, хто може мати доступ до файлів. Ця модель заснована на системі міток безпеки (security label).

► Однак MAC має дуже централізований характер, де для кожного доступу суб'єкт повинен отримати дозвіл від центру адміністрування. У цій перспективі MAC не забезпечує детальний контроль доступу до ресурсів у **великомасштабній системі**

Моделі управління доступом

► Коли система приймає рішення на доступ до об'єкта, вона ґрунтується на рівні допуску суб'єкта, класифікації об'єкта і політики безпеки системи. (Моделі Бела – ЛаПадуло,……..)

► Користувачі отримують рівні допуску (секретно, цілком таємно, конфіденційно і т.д.), таким же способом класифікуються дані. Допуски і класи даних зберігаються в **мітках безпеки**.

► **Структура мітки** . Класифікація використовує ієрархічну структуру, в якій один рівень є більш довіреною, ніж інший. Категорії не використовують ієрархічну структуру, вони представляють собою окремі види інформації в рамках системи. Категорії можуть відповідати структурі підрозділів компанії, проектам або рівнями посад. Класифікація проводиться за ступенем конфіденційності інформації, вона залежить від середовища, в якій працює компанія.

Мітка об'єкту

рівень	Область
Таємно	ВМС України

Мітка суб'єкту

рівень	Область
Таємно	ВПС України

Рольове управління доступом

► Рольове управління доступом являє собою розвиток політики дискреційного розмежування доступу, при цьому права доступу суб'єктів системи на об'єкти групуються з урахуванням специфіки їх застосування, утворюючи ролі.

► У RBAC кожному користувачеві присвоюється певний **набір ролей**, а **дозволи** (тобто санкціоновані дії) **присвоюються ролям** на основі рішень щодо політики.

► Роль визначає функцію в системі в ієрархічному порядку.

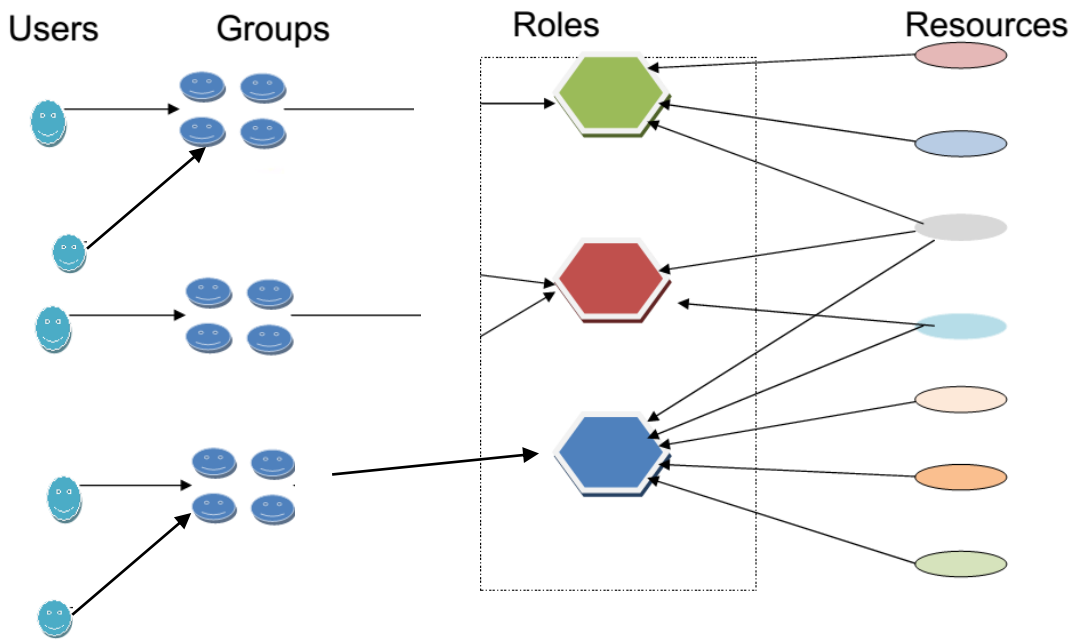
► Взаємозв'язок між користувачами та ролями - це відображення відношення "багато-до-багатьох", де користувачеві може бути призначено кілька ролей, а роль може мати декількох користувачів.

► У RBAC, якщо роль користувача змінюється, змінюються відповідні дозволи, доступні для цього користувача. Це означає, що доступ користувача може бути відрегульований шляхом повторного призначення членства в ролі і без зміни призначення дозволів. Це забезпечує більшу **гнучкість** у **спрощенні адміністрування** безпеки для великих організацій, які повинні керувати своїми ресурсами на основі відповідальності та кваліфікації користувача.

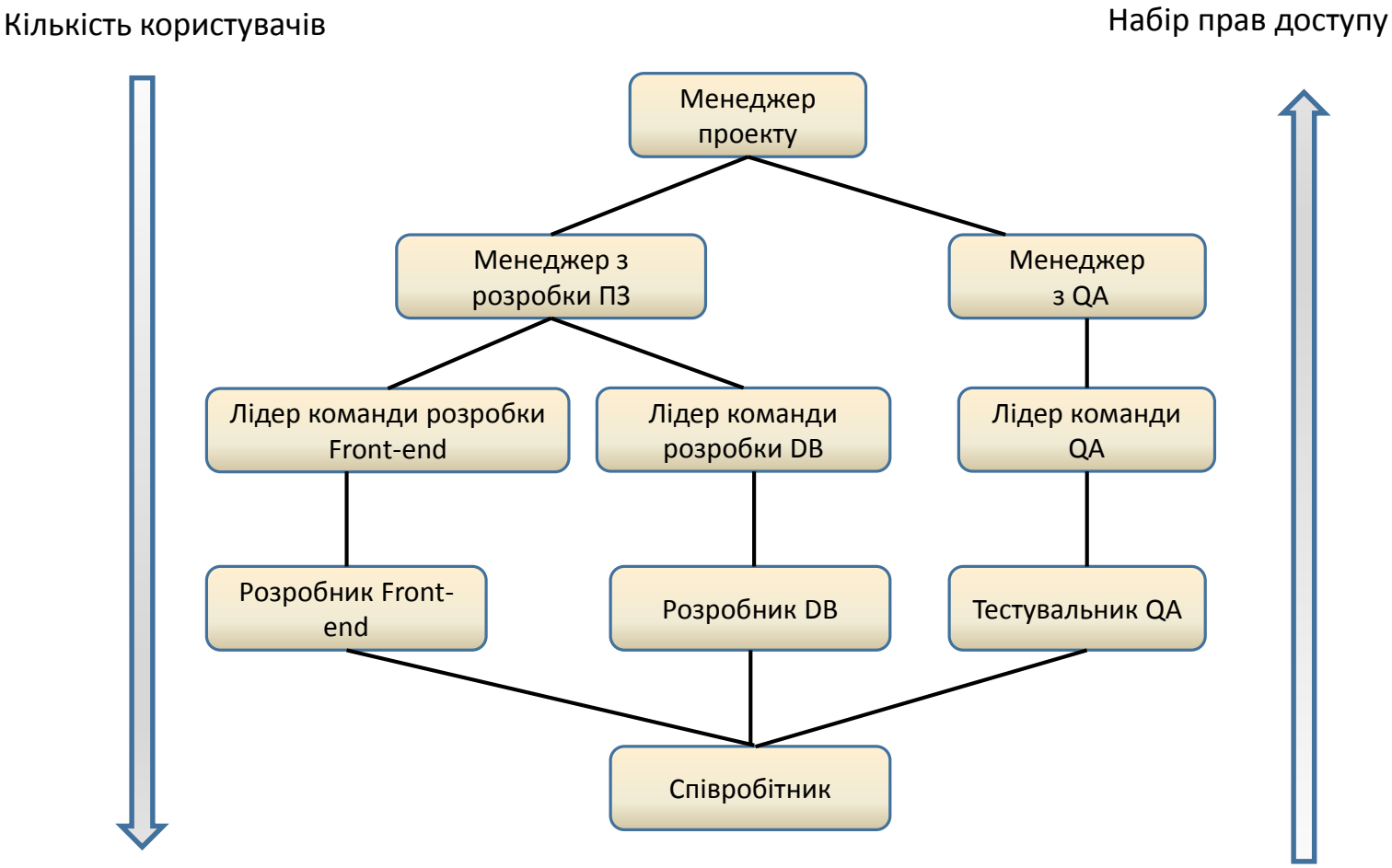
► Крім того, RBAC добре підходить для вимог „розділення обов’язків”, де всі дозволи не призначені одному користувачеві для прийняття рішення

Моделі управління доступом

Рольове управління доступом



Наслідування прав доступу



► Адміністратори визначають ролі (і групи) за посадовими інструкціями або робочими функціями. У багатьох випадках це відповідає ієрархії організації, задокументованої в організаційній діаграмі. Користувачі, які займають керівні посади, матимуть більший доступ до ресурсів, ніж користувачі на тимчасовій роботі

Рольове управління доступом

► Взаємозв'язки "багато-до-багатьох" між користувачами та ролями, а також між ролями та дозволами забезпечують гнучкість та деталізацію призначення, яких немає в звичайних схемах DAC .

► Без цієї гнучкості та деталізації існує більший ризик того, що користувачеві може бути надано більше доступу до ресурсів, ніж це потрібно через обмежений контроль над типами доступу, які можна дозволити.

► Документ NIST RBAC наводить такий приклад, який неможна реалізувати за допомогою DAC : Користувачам можуть дивиться зміст каталогів та модифікувати існуючі файли. Але не мають можливості створювати нові файли. Вони можуть додавати нові записи, але не можуть модифікувати існуючі

► Рольовий доступ має одну стратегічну адміністративну слабкість. Розповзання привілеїв - непотрібне, часто погано обґрунтоване та потенційно небезпечне накопичення привілеїв доступу, які більше не потрібні для виконання обов'язків. Щоб стримати зростання привілеїв, необхідно переглядати привілеї доступу кожного працівника у світлі його поточних обов'язків на регулярній періодичній основі.

Рольове управління доступом

► RBAC корисний в динамічних середовищах з частими змінами персоналу, оскільки адміністратори можуть легко надати кілька дозволів, просто додавши нового користувача у відповідну роль. Варто зазначити, що користувачі можуть належати до кількох ролей або груп. Наприклад, використовуючи предметну область банк, менеджери можуть належати до ролі менеджерів, ролі кредитних спеціалістів і ролі касирів

► Microsoft реалізують RBAC з використанням груп. Деякі групи, наприклад група локальних адміністраторів, є попередньо визначеними. Однак адміністратори можуть створювати додаткові групи, щоб відповідати посадовим функціям або ролям, які використовуються в організації

RULE BASED ACCESS CONTROL

► **Маршрутизатори, брандмауери**

An additional rule clause to deny access

```
<rule order="allow,deny">
  <precondition>
    <predicate>
      from("10.0.0.24")
    </predicate>
  </precondition>
</rule>
```

XACML (eXtensible Access Control Markup Language)

```
<rule order="allow,deny">
 <precondition>
  <predicate>
   ${Args::OP} eq "CREATE_OP"
  </predicate>
 </precondition>

 <allow>
   dacs_admin()
 </allow>
 </rule>
```

► Доступ дозволений або обмежений для об'єктів в залежності від **часу доби**, в залежності від **маркування ресурсу** і **облікових даних користувача** (визначається політикою).

► Доступ дозволений або обмежений для об'єктів на основі характеристик безпеки HTTP-клієнта (наприклад, довжина біта SSL, **інформація про версії**, **вихідний IP-адреса** або домен і т. Д.).

RULE BASED ACCESS CONTROL

► Модель контролю доступу на основі правил використовує набір правил, обмежень або фільтрів, щоб визначити, що може, а що не може відбуватися в системі. Це включає надання суб’єкту доступу до об’єкта або надання суб’єкту можливості виконувати дію. Відмінною характеристикою моделей контролю доступу на основі правил є те, що вони мають глобальні правила, які застосовуються до всіх суб'єктів.

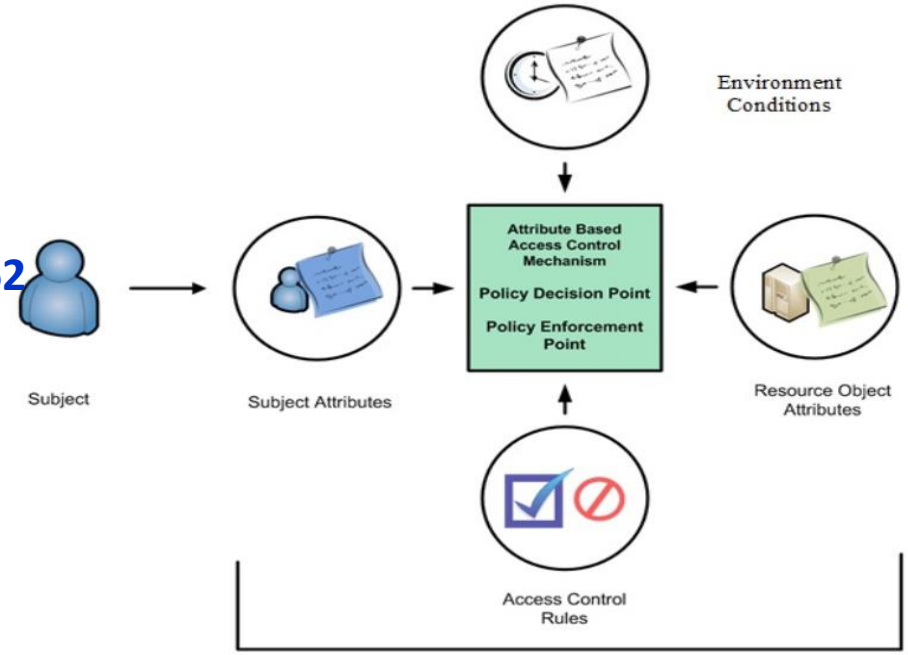
► Одним із поширених прикладів моделі контролю доступу на основі правил є брандмауер. Брандмауери включають набір правил або фільтрів у ACL, визначених адміністратором. Брандмауер перевіряє весь трафік, що проходить через нього, і дозволяє лише трафік, який відповідає одному з правил. Брандмауери включають останнє правило (іменується як правило неявного заборони), що забороняє або блокує весь інший трафік. Початкові правила визначають трафік, який дозволить брандмауер. Правило неявного заборони забороняє весь інший трафік.

► Це останнє правило іноді доступне для перегляду в ACL, щоб ви могли його побачити. В інших випадках правило неявного заперечення мається на увазі як остаточне правило, але явно не зазначено в ACL.

Attribute-Based Access Control (ABAC)

► **Контроль доступу на основі атрибутів (ABAC)** базується на конкретних атрибутах **людини, ресурсу або середовища**. Її часто можна зустріти в інфраструктурних системах, таких як мережеві або телекомунікаційні середовища.

NIST Special Publication 800-162



Attribute-Based Access Control (ABAC)

► У системі контролю доступу на основі атрибутів (ABAC) дозволи контролю доступу призначаються на основі атрибутів **людини, ресурсу або середовища**.

► Ці атрибути можна розглядати як властивості, що описують специфічні особливості предметів, об'єктів, середовищ, умов тощо.

► Наприклад, **ім'я та вік є атрибутами суб'єкта**, де **час і місцезнаходження є атрибутами** навколишнього середовища.
Написані політики для призначення дозволів на доступ на основі параметрів атрибута.

► Зауважте, що ABAC таким чином дозволяє створювати правила доступу, не створюючи явного зіставлення користувачів з дозволами. Це може забезпечити значний ступінь гнучкості порівняно з RBAC. У ABAC рішення щодо доступу можуть **змінюватися між запитами**, просто змінюючи значення атрибутів. Немає необхідності змінювати заздалегідь визначені відносини суб’єкта та об’єкта

Attribute-Based Access Control (ABAC)

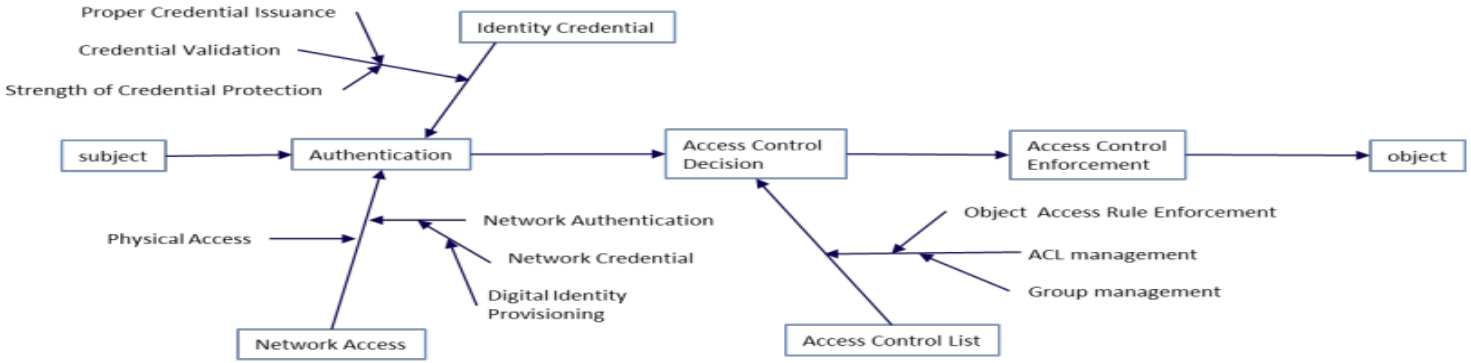
► **Предметні** атрибути належать **особі**. Прикладом предметних атрибутів є CAPTCHA. CAPTCHA контролюють доступ на основі того, чи може сторона з іншого боку пройти такий тест (теоретично) занадто складний для машини, щоб добудувати. Атрибути, **яки надаються співробітнику при прийомі на роботу**

► **Атрибути ресурсу** належать такому ресурсу, як **операційна система або програма**. Наприклад, деяке програмне забезпечення працює лише на певній операційній системі, а деякі веб-сайти працюють лише з певними браузерами. Можна застосувати цей тип контролю доступу як засіб безпеки, вимагаючи від когось використовувати певне програмне забезпечення або протоколи для зв'язку.

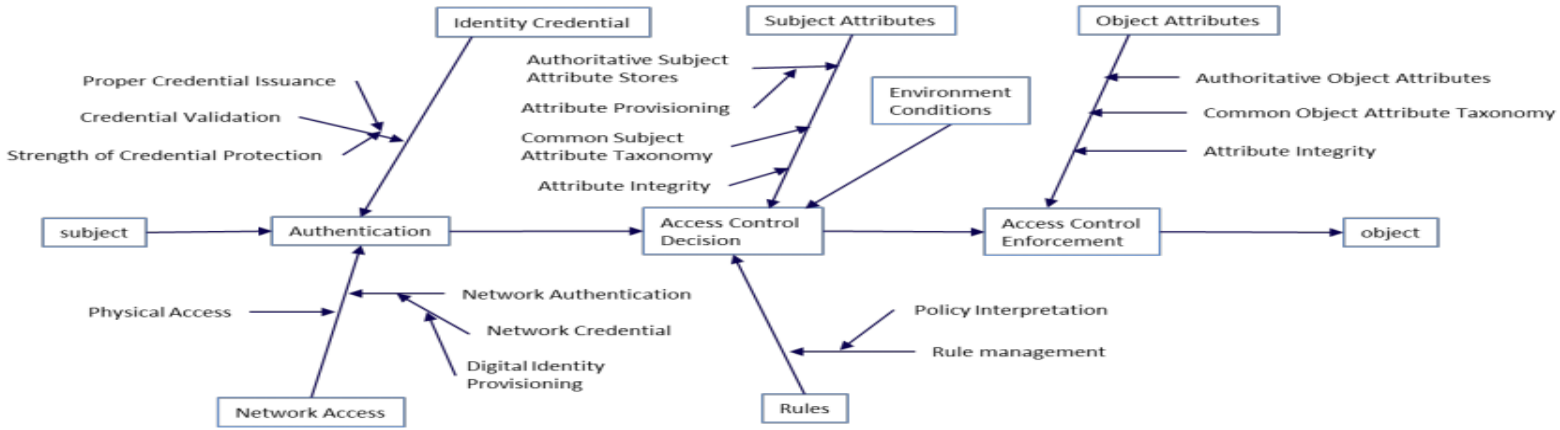
► **Атрибути об'єкта** не повинні змінюватися суб'єктом для маніпулювання результатом рішення контролю доступу. Атрибути об’єкта повинні бути доступними для отримання за допомогою механізмів контролю доступу для прийняття рішень щодо контролю доступу

Attribute-Based Access Control (ABAC)

ACL Trust Chain

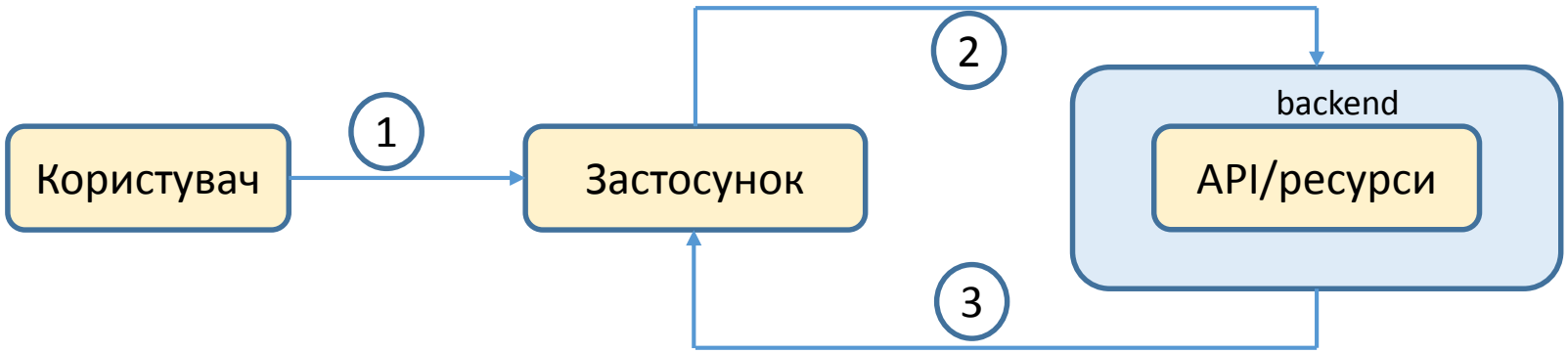


ABAC Trust Chain



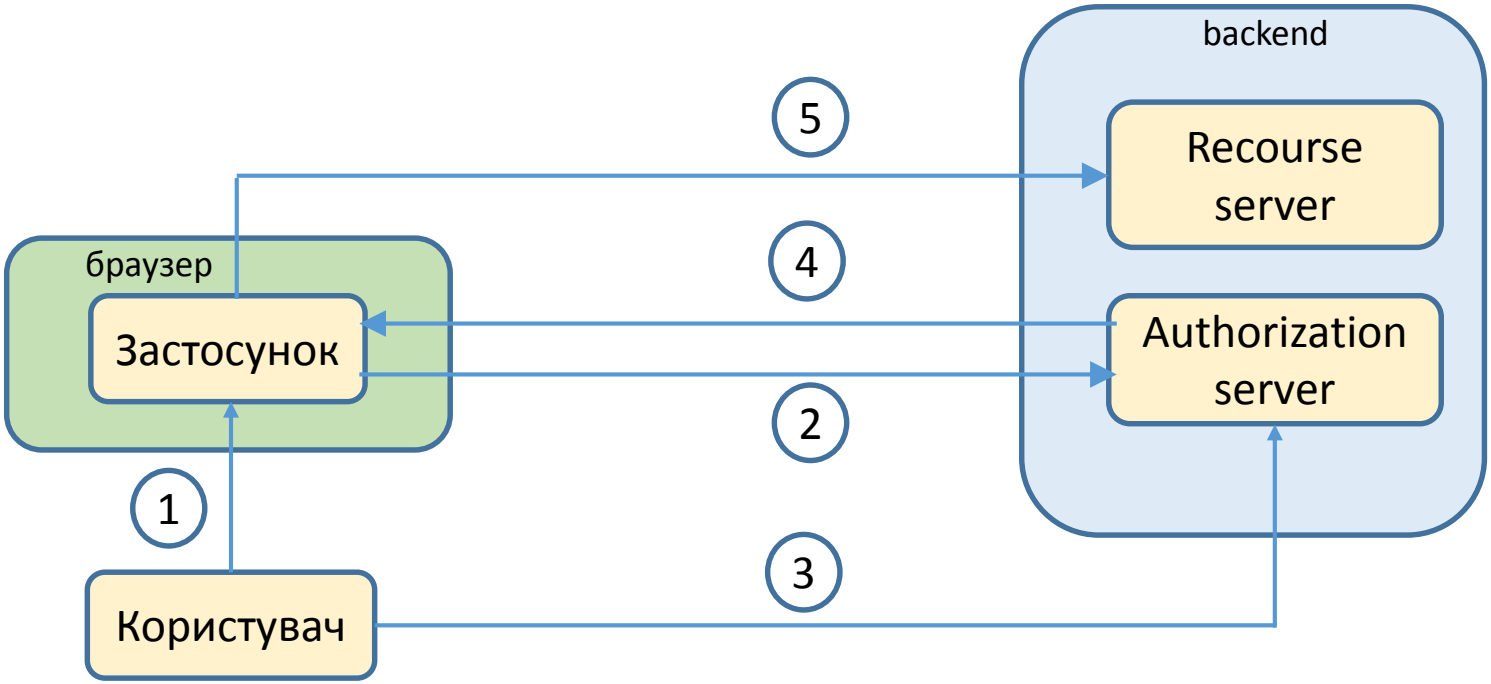
Автентифікація OAuth 2.0

- стандартний спосіб надання доступу (без OAuth)



1. Користувач надає повноваження застосунку (вказує облікові дані)
2. Застосунок використовує повноваження для доступу до API або ресурсів серверу
3. Застосунок використовує ресурси/API без обмежень, як користувач

Автентифікація OAuth 2.0



1. Користувач ініціює доступ до ресурсу/ API через застосунок
2. Застосунок звертається до серверу авторизації
3. Серверу авторизації через браузер автентифікує користувача (він вказує облікові дані)
4. Сервер авторизації надає застосунку токен доступу.
5. Застосунок використовує токен для доступу до API або ресурсів серверу

Типові недоліки механізму управління доступом

- Жорстко запрограмовані перевірки ролей у коді програми

```
if ((user.isManager()
{
//execute action
}
```

```
if ((user.isManager()   ||
user.isAdministrator() ||
user.isEditor()        ||
user.isUser() && user.id() != 1132))
{
//execute action
}
```

- Ускладнює аудит і тестування
- Щоразу, коли політику контролю доступу потрібно змінити, потрібно ввести новий код
- RBAC часто недостатньо гранульований (горизонтальне управління доступом)
- Легко помилятися з перевірками в коді

Типові недоліки механізму управління доступом

- Відсутність логіки централізованого контролю доступу
- Ненадійні дані сприяють прийняттю рішень щодо контролю доступу
- Контроль доступу, який «відкритий за замовчуванням»
- Відсутність вирішення горизонтального контролю доступу в стандартизований спосіб
- Логіка контролю доступу, яку потрібно виконувати вручну додається до кожної кінцевої точки в коді
- Контроль доступу, який вимагає політики для кожного користувача

Ніколи не покладайтеся на ненадійні дані

► Ніколи не довіряйте даним запиту для прийняття рішень щодо контролю доступу

► Ніколи не приймайте рішення про управління доступом в скриптах JavaScript

► Ніколи не приймайте рішення про авторизацію, ґрунтуючись виключно на:

- приховані поля
- значення файлів cookie
- параметри форми
- параметри URL-адреси
- будь-що інше із запиту

► Ніколи не залежати від порядку значень, надісланих від клієнта

Авторизація і управління доступом в ВЕБ – застосунках

Авторизація і управління доступом

► Як правило, в веб-додатках реалізується рольова (**RBAC**) або атрибутна (**ABAC**) політика управління доступом.

► Механізм авторизації рекомендується реалізовувати на рівнях веб-застосунку:

- **представлення** (інтерфейс, форми введення і виведення інформації),

- **бізнес-логіки** (функціонал)

- **Даних** (середовища зберігання і передачі даних)

Авторизація і управління доступом

► **Рівень представлення** – обмеження інтерфейсу (наприклад, форми, фрейми, посилання, кнопки), на який користувач не має прав доступу.

► **Рівень бізнес-логіки** забезпечує виконання перевірки наявності відповідних прав доступу до виконання запиту в веб-додатку, тобто ніякі функції **не можуть бути виконані до авторизації** (наприклад, якщо користувач відправляє запит на видалення облікового запису, то веб-додаток повинен переконатися, що користувач має право на видалення облікового запису і не виконувати ніяких функцій до того, як це буде встановлено).

► **Рівень даних** забезпечує перевірку наявності прав доступу користувача до даних, а не тільки до функціоналу обробки даних (наприклад, користувач, використовуючи URL виду

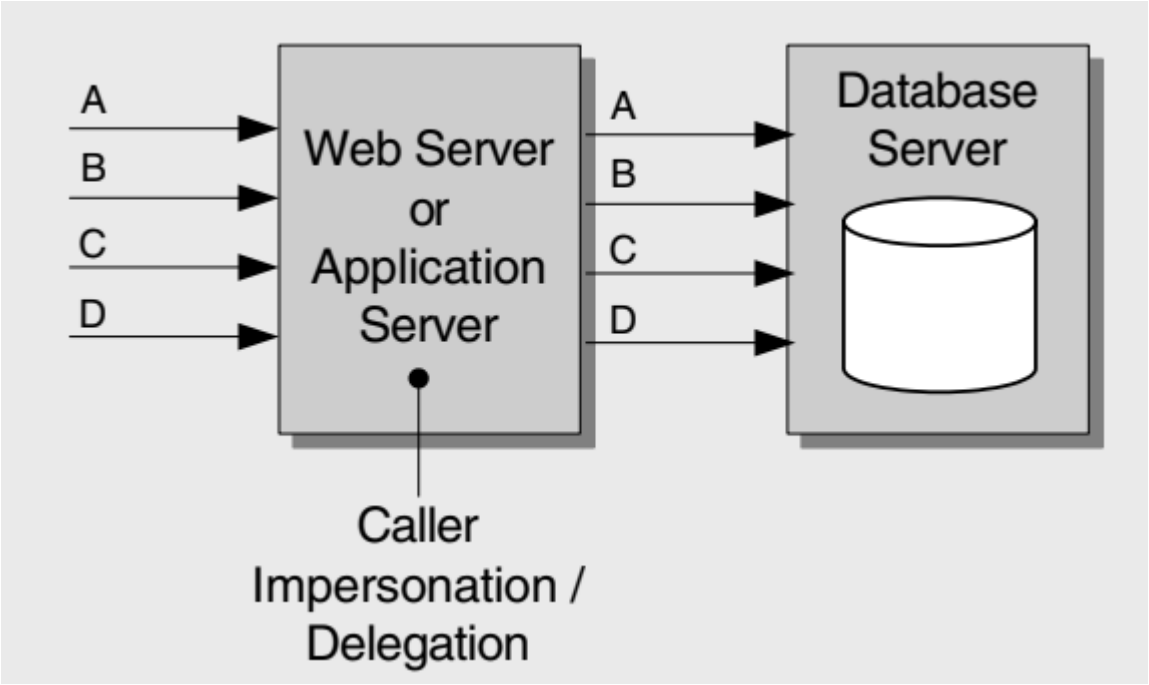
/delete?Record=1

повинен видаляти тільки ті записи в базі даних, на які він має право доступу DELETE) .

Рівень деталізації дозволів

► **Найбільш детальний підхід**

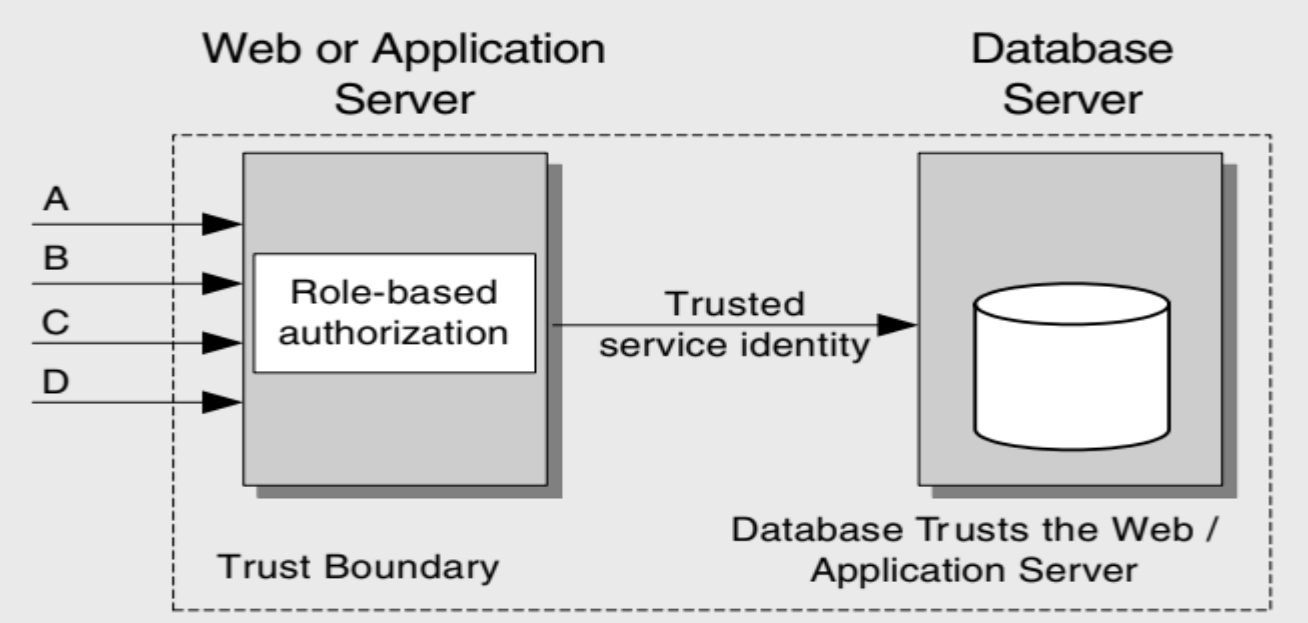
- Обмежена масштабованість
- найчастіше зустрічається в обмежених програмах на базі інтрамережі



Рівень деталізації дозволів

► Модель довіреного сервера

- Найменш детальний, але найбільш масштабований підхід



- Дозволи, надані програми в базі даних, є загальними, незалежно від ідентичності абонента. Доступ до класів та методів обмежений залежно від **членства в ролі**.

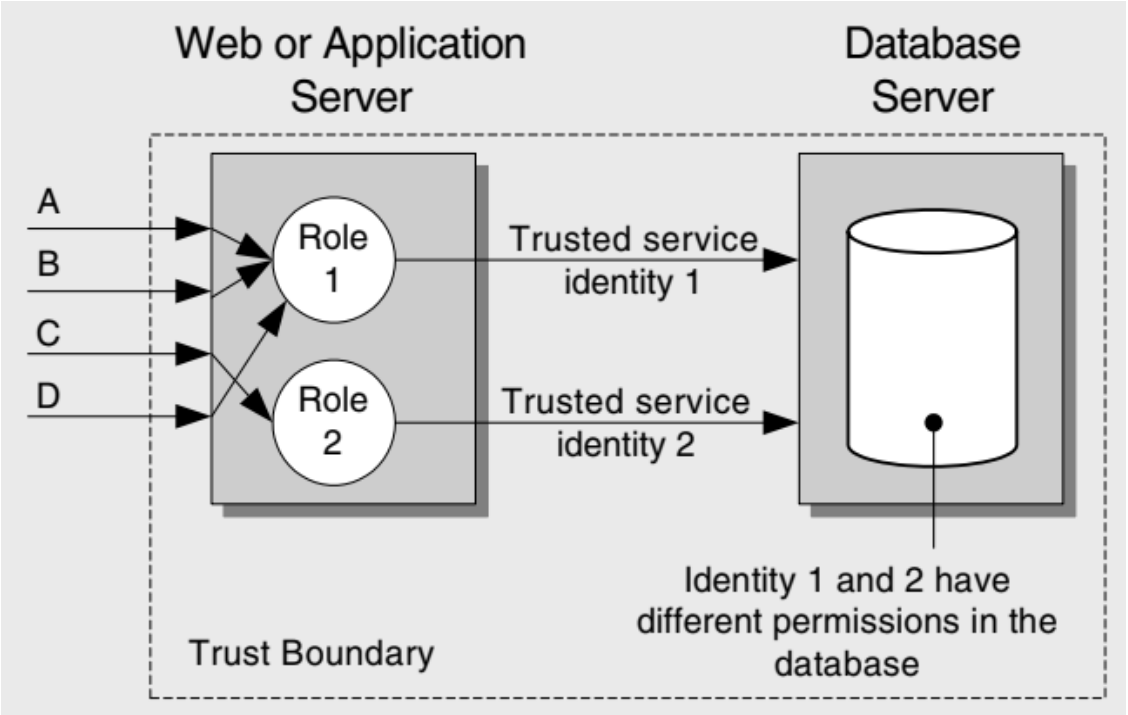
Для отримання даних для кожного користувача можна передати оригінальний ідентифікатор абонента бази даних на рівні застосунку через параметри, наприклад:

```
SELECT field1, field2, field3 FROM Table1 WHERE {some search criteria} AND UserName = @originalCallerUserName
```

Рівень деталізації дозволів

► Гібридна модель

- використанні обмеженого набору ідентифікаторів для доступу до ресурсів на **основі членства в ролі**. Користувачі відображаються на ролях, а доступ до класів та методів обмежений залежно від членства в ролях



Впровадження контролю доступу

► приклад додавання перевірки дозволів до функцій **Python**

```
from django.contrib.auth.decorators import login_required,
permission_required
(1) @login_required
(2) @permission_required ('content.can_publish')
def pub_post (запит):
# Опублікуйте допис на першій сторінці
```

Веб-сервер вимагає, щоб користувач увійшов до системи **(1)** і мав дозволи на публікацію вмісту **(2)**, перш ніж дозволити йому публікувати публікацію.

Перевірка дозволу вбудована в **Ruby**, використовуючи АПІ бібліотеки **pundit**.

```
def publish
@post = Post.find(params[:id])
(1) authorize @post, :update?
@post.publish!
redirect_to @post
end
```

Виклик методу **(1)** перевірки у бібліотеки, чи має користувач, що ввійшов у систему, дозвіл на оновлення публікації в соціальній мережі, описаної об’єктом @post.

Уразливості контролю доступу та підвищення привілеїв

► **Горизонтальне підвищення привілеїв**: відбувається, коли користувач може виконати дію або отримати доступ **до даних іншого користувача з таким же рівнем дозволів**.

► **Вертикальне підвищення привілеїв**: відбувається, коли користувач може виконати дію або отримати **доступ до даних**, для яких потрібно **рівень доступу**, **що перевищує його роль.**
Наприклад, якщо користувач без прав адміністратора може фактично отримати доступ до сторінки адміністратора, де він може видалити облікові записи користувачів, то це вертикальне підвищення привілеїв
► **Broken access control** - Зламані засоби контролю доступу - це часто зустрічається і часто **критична** уразливість безпеки. Зламаний контроль доступу **займає 5 місце** в списку 10 найбільш вразливих місць веб-додатків **OWASP** 2017 року.

► Проблеми контролю доступу зазвичай **не виявляються динамічним скануванням** вразливостей і інструментами **статичного аналізу** вихідного коду, оскільки вони вимагають розуміння того, як певні фрагменти даних використовуються в веб-додатку. **Ручне тестування** - кращий спосіб виявити відсутні або зламані засоби контролю доступу (наприклад **диференціальний аналіз**)

Вертикальне підвищення привілеїв

► **Незахищені функціональні можливості**

- Наприклад, адміністративні функції можуть бути пов'язані зі стартовою сторінкою адміністратора, але не зі стартовою сторінкою користувача. Однак користувач може просто отримати доступ до адміністративних функцій, перейшовши безпосередньо до відповідного URL-адресою адміністратора

- У деяких випадках конфіденційна функціональність не захищена надійно, а ховається за рахунок надання їй менш передбачуваного URL-адреси: так звана **безпека по непомітності**

https://insecure-website.com/administrator-panel-**yb556**

Така URL-адреса може бути розкритий в JavaScript, який доступний в інтерфейсі:

```
<Script>
create var isAdmin = false;
if (isAdmin) {
...
var adminPanelTag = document.createElement ( 'a');
adminPanelTag.setAttribute ( 'https://insecure-
website.com/administrator-panel-yb556');
adminPanelTag.innerText = 'Панель адміністратора';
...
}
```

Вертикальне підвищення привілеїв

► Порушення контролю доступу через неправильну конфігурації платформи

Деякі додатки застосовують контроль доступу на рівні платформи, обмежуючи доступ до певних URL-адресами і методам HTTP в залежності від ролі користувача. Наприклад, додаток може налаштувати правила, подібні наступним:

```
DENY: POST, /admin/deleteUser, manager
```

Деякі платформи додатків підтримують різні нестандартні заголовки HTTP, які можна використовувати для перевизначення URL-адреси в вихідному запиті, наприклад **X-Original-URL** і **X-Rewrite-URL**. Якщо додаток дозволяє перевизначити URL-адресу за допомогою заголовка запиту, то можна обійти елементи управління доступом за допомогою запиту, подібного наступного:

```
POST / HTTP / 1.1
X-Original-URL: /admin/deleteUser
...
```

Горизонтальне підвищення привілеїв

► Атаки горизонтального підвищення привілеїв можуть використовувати методи **експлойта**, аналогічні методам вертикального підвищення привілеїв.
Наприклад, користувач зазвичай може отримати доступ до сторінки свого облікового запису, використовуючи наступний URL-адресу:

`https://insecure-website.com/myaccount?id=123`

Тепер, якщо зловмисник змінює на id **іншого користувача**, тоді зловмисник може отримати доступ до сторінки облікового запису іншого користувача з пов'язаними даними і функціями.

► У деяких додатках експлуатований параметр **не має передбачуваного значення**.
Наприклад, замість зростаючого числа додаток може використовувати глобально унікальні ідентифікатори (GUID) для ідентифікації користувачів.
Однак ідентифікатори GUID, що належать іншим користувачам, можуть бути розкриті в іншому місці додатка, на яке посилаються користувачі, **наприклад в повідомленнях користувачів**.
У деяких випадках додаток виявляє, коли користувачеві не дозволено доступ до ресурсу, і перенаправляє на сторінку входу. Однак відповідь, що містить перенаправлення, може як і раніше містити деякі конфіденційні дані, що належать цільовим користувачеві, тому атака все ще успішна

Контроль доступу на основі Referer

► Деякі веб-сайти засновують елементи управління доступом на **Referer**, заголовку представленому в HTTP-запиті.

Заголовок **Referer** як правило додають до запитів браузерів **щоб вказати сторінку з якої був ініційований запит**.

Наприклад, припустимо, що додаток надійно забезпечує контроль доступу до головної адміністративної сторінці в **/admin**, але для таких вкладених сторинок, як **/admin/deleteUser**, перевіряється тільки **Referer** заголовок. Якщо Referer заголовку містить основний URL-адресу **/admin**, запит дозволяється.
У цій ситуації, оскільки Referer заголовок може повністю контролюватися зловмисником, він може **підробляти прямі запити до конфіденційних** підлеглих сторинок, надаючи необхідний Referer заголовок, і таким чином отримувати несанкціонований доступ.

Жорстко закодовані ролі

- ускладнює «перевірку» політики додатки для аудиту або з метою забезпечення якості.
- У будь-який час, коли потрібно змінити політику контролю доступу, потрібно створити новий код.
- RBAC часто недостатньо деталізований.
- Легко робити помилки

```
void editProfile(User u,
EditUser eu)
if (u.isManager()) {
editUser(eu)
}
```

```
if ((user.isManager() ||
user.isAdministrator() ||
user.isEditor() ||
user.isUser() &&
user.id() != 1132))
{
//execute action
}
```

Запобігання уразливості доступу

Ніколи не покладайтеся на ненадійні дані

- Ніколи не довіряйте даними запиту для рішень контролю доступу
- Ніколи не приймайте рішення про управління доступом в JavaScript (**на клієнті**)
- Ніколи не приймайте рішення про авторизацію виключно на основі

- приховані поля

- значення cookie

- параметри форми

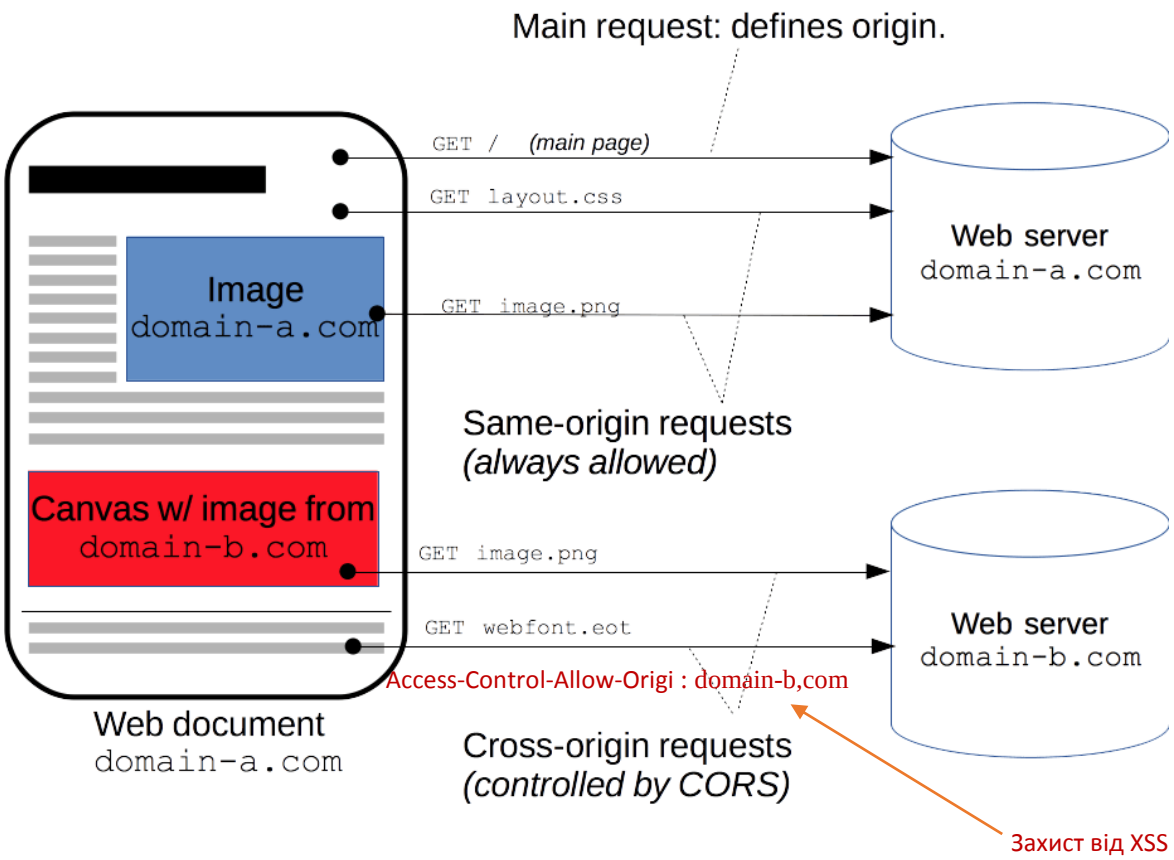
- параметри URL

- що-небудь ще з запиту

- Ніколи не залежати від **порядку значень**, відправлених від клієнта

Запобігання уразливості доступу

- Ніколи не покладайтеся **лише на заплутування** для контролю доступу.
- Якщо ресурс не призначений для публічного доступу, **за замовчуванням забороніть доступ**.
- Скрізь, де це можливо, використовуйте **єдиний механізм** в масштабі всього застосунку для забезпечення контролю доступу.
- **На рівні коду** зробіть **обов'язковим** для розробників оголошувати доступ для кожного ресурсу, і за замовчуванням забороняти доступ.
- **Ретельно перевіряйте** засоби контролю доступу, щоб переконатися, що вони працюють так, як задумано.
- Додаткові кроки можуть включати обмеження доступу до списків каталогів анулювання маркерів автентифікації при виході з системи

CORS

- **Same Origin Policy – SOP**
- **Cross-Origin Resource Sharing** (**CORS**, спільне використання ресурсів з різних джерел) — механізм безпеки сучасних браузерів, який дозволяє вебсторінкам використовувати дані, що знаходяться на інших доменах. Тобто реалізує захист для вебсторінок, які не відповідають політиці SOP.